

Project Part II

Docker Swarm and Resilient Kubernetes on Azure

Flash Tans: a resilient, cost-effective cloud-native platform

Group: Kane

TEAM MEMBER	ANDREW ID
Hastings Tugume	htugume
Sifen Gebremariam	sgebrema
Davy Mbuto Nkurunziza	davymbun



Table of Contents

1. Introduction	2
1.1 Team and responsibilities	2
1.2 Screencast link	2
2. Architecture Overview	3
2.1 Azure resources	3
3. Part 1: Architecting the Foundation on Azure	4
3.1 Virtual Network with public and private subnets	4
3.2 Network Security Group	4
3.3 Azure Kubernetes Service cluster	5
3.4 Load Balancer and monitoring	6
3.5 VNet peering	8
4. Part 2: Optimizing the Application Container	9
4.1 Multi-stage Dockerfile	9
4.2 Build and push	9
5. Part 3: Simulating the Stack with Docker Compose	11
5.1 docker-compose.yml	11
5.2 Launch and persistence test	11
6. Part 4: Deploying with Docker Swarm on Azure	13
6.1 Initializing the swarm	13
6.2 Adapting the Compose file for Swarm	13
Figure 19: Swarm Compose file with Docker secrets, overlay network and MySQL healthcheck	13
6.3 Deploying from the local Docker client	14
6.4 Load balancer access	15
7. Part 5: High Availability with Kubernetes and MySQL Replication	16
7.1 MySQL Group Replication	16
7.2 Kubernetes manifests	18
7.3 Local deployment on Minikube	18
7.4 Migration to AKS	19
7.5 Resilience testing: the flash sale simulation	21
8. Swarm vs. Kubernetes: Deployment Experience	27

1. Introduction

Flash Tans is an e-commerce startup whose flash sales cause traffic spikes that crash its website. This project containerizes the Flash Tans Node.js application, adds a highly available MySQL data layer, and orchestrates the stack with Docker Compose, Docker Swarm and Azure Kubernetes Service (AKS). The goal is zero downtime during peak sales while keeping costs low during quiet periods.

1.1 Team and responsibilities

Team member	Andrew ID	Main responsibilities
Hastings Tugume	htugume	Azure infrastructure, networking, Load Balancer
Davy Mbuto Nkurunziza	davymbun	AKS, MySQL Group Replication, Kubernetes manifests
Sifen Gebremariam	sgebrema	Dockerfile, Docker Compose, Azure docker swarm

1.2 Screencast link

Video demonstration:

<https://drive.google.com/file/d/12LuB8C656U5tIs17PMNBV9ed0E4r6X0d/view?usp=sharing>

2. Architecture Overview

The solution runs in two Azure environments connected by VNet peering. The Docker Swarm cluster runs on three virtual machines in flashtans-vnet, with the database node placed in a private subnet that has no public IP. The AKS cluster runs in its own VNet and hosts the final production deployment.

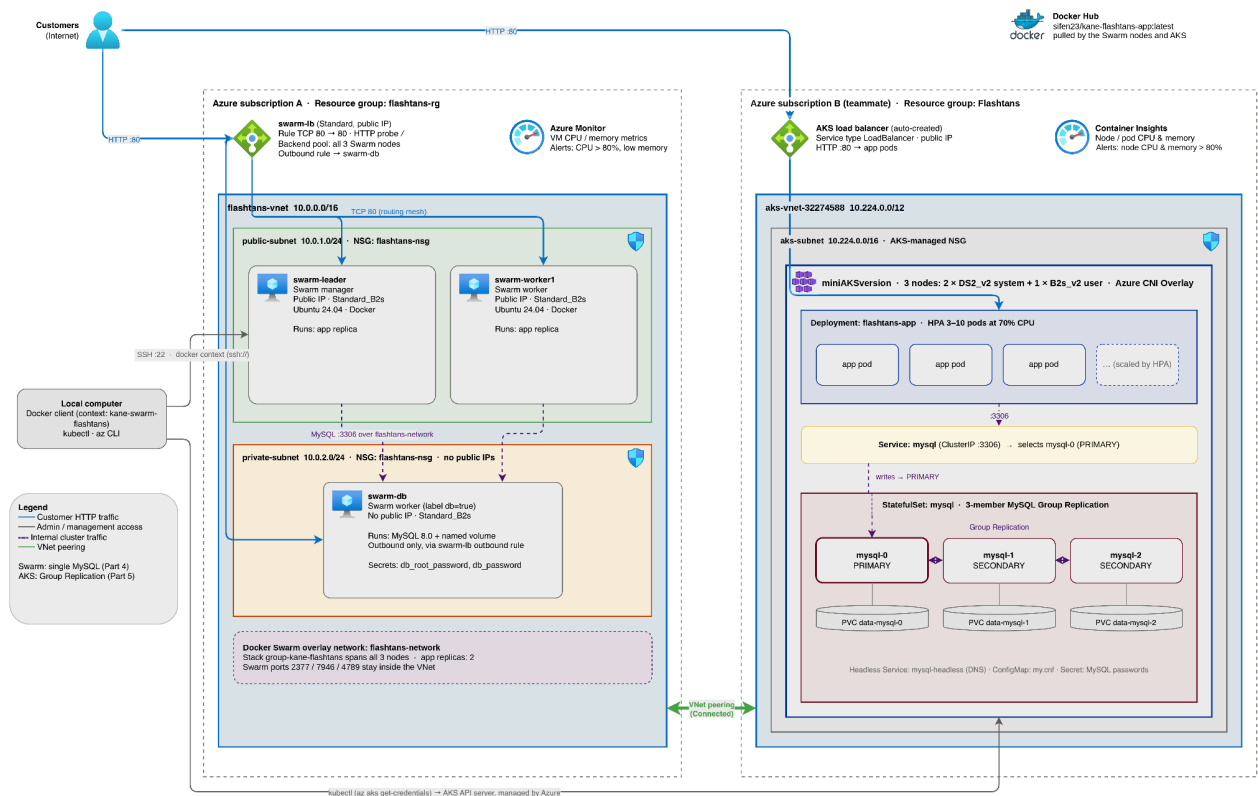


Figure 1: Overall architecture of the Flash Tans deployment

2.1 Azure resources

Resource	Location / range	Purpose
flashtans-vnet	10.0.0.0/16	Network for the Docker Swarm environment
public-subnet	10.0.1.0/24	swarm-manager (hostname swarm-leader) and swarm-worker1 (public IPs)
private-subnet	10.0.2.0/24	swarm-db, no public IP, isolated from the internet
swarm-lb	Public, Standard SKU	Distributes HTTP traffic to the Swarm nodes; outbound access for swarm-db

3. Part 1: Architecting the Foundation on Azure

3.1 Virtual Network with public and private subnets

A VNet was created with a public subnet for internet-facing nodes and a private subnet for the database. Resources in the private subnet have no public IP, so they cannot be reached directly from the internet. swarm-db receives outside traffic only through the load balancer on port 80, where the Swarm routing mesh forwards each request to an app replica; MySQL (3306) is allowed only from within the VNet and the peered AKS VNet.

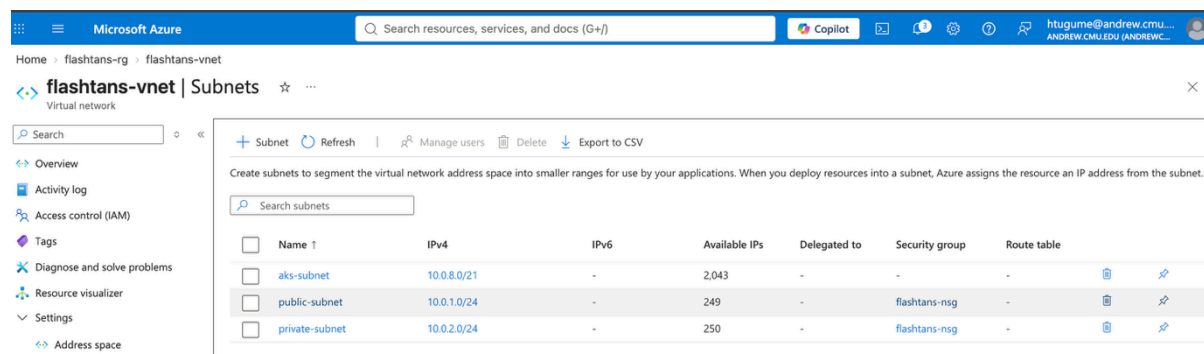


Figure 2: flashtans-vnet with public and private subnets

3.2 Network Security Group

An NSG was attached to both subnets with the inbound rules below. Internal ports are restricted to the VirtualNetwork service tag rather than the internet.

Rule	Source	Ports / protocol	Purpose
flashtans-ssh	Any	22 / TCP	Administrative access
flashtans-http	Any	80 / TCP	Public application traffic
flashtans-https	Any	443 / TCP	Public application traffic
flashtans-ds-tcp	VirtualNetwork	2377, 7946 / TCP	Swarm management and node communication
flashtans-ds-udp	VirtualNetwork	7946, 4789 / UDP	Swarm gossip and overlay network
flashtans-k8s	VirtualNetwork	6443 / TCP	Kubernetes API server
flashtans-mysql	VirtualNetwork	3306 / TCP	Database traffic

Note: Azure's default rule AllowVnetInBound already permits all traffic within a VNet and between peered VNets. The Swarm, Kubernetes API and MySQL rules are therefore technically redundant but were added to VirtualNetwork for clarity, while only 22, 80 and 443 strictly require explicit rules. SSH (22) is open to any source for team access during the project; in production it would be restricted to known IP ranges or replaced with Azure Bastion.

Priority	Name	Port	Protocol	Source	Destination	Action
100	flashtans-ssh	22	TCP	Any	Any	Allow
110	flashtans-http	80	TCP	Any	Any	Allow
120	flashtans-https	443	TCP	Any	Any	Allow
130	flashtans-ds-tcp	2377,7946	TCP	VirtualNetwork	VirtualNetwork	Allow
140	flashtans-ds-udp	7946,4789	UDP	VirtualNetwork	VirtualNetwork	Allow
150	flashtans-k8s	6443	TCP	VirtualNetwork	VirtualNetwork	Allow
160	flashtans-mysql	3306	TCP	VirtualNetwork	VirtualNetwork	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancer...	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny

Figure 3: NSG inbound rules

Name	Address range	Virtual network
private-subnet	10.0.2.0/24	flashtans-vnet
public-subnet	10.0.1.0/24	flashtans-vnet

Figure 4: NSG associated with the public and private subnets

3.3 Azure Kubernetes Service cluster

A standard AKS cluster (miniAKSversion) with three nodes was provisioned with Azure CNI Overlay in its own VNet, with Container Insights enabled. The AKS cluster was configured with three worker nodes distributed across two node pools. The system node pool contains two Standard_DS2_v2 nodes and primarily hosts Kubernetes system components and cluster services. A separate user node pool contains one Standard_B2s_v2 node and provides additional capacity for application workloads. Although the nodes belong to different pools and use different VM sizes, all three are part of the same AKS cluster and are managed by the same Kubernetes control plane. Kubernetes schedules application pods across the available cluster capacity, while the flashtans-app LoadBalancer Service provides a single public endpoint and routes incoming traffic to the healthy

application pods regardless of which node they are running on.

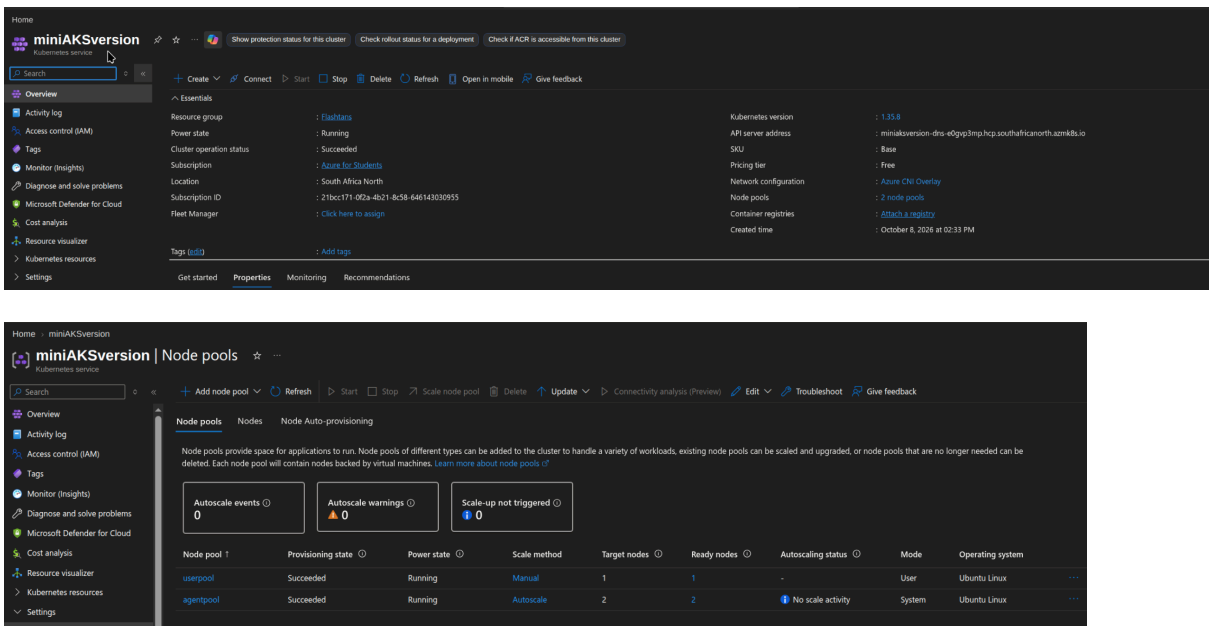


Figure 5: AKS cluster overview

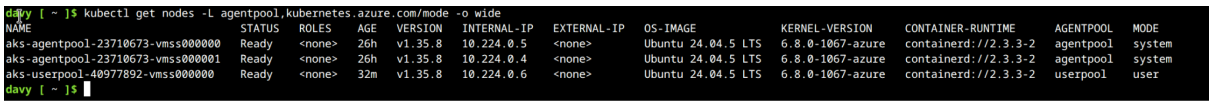
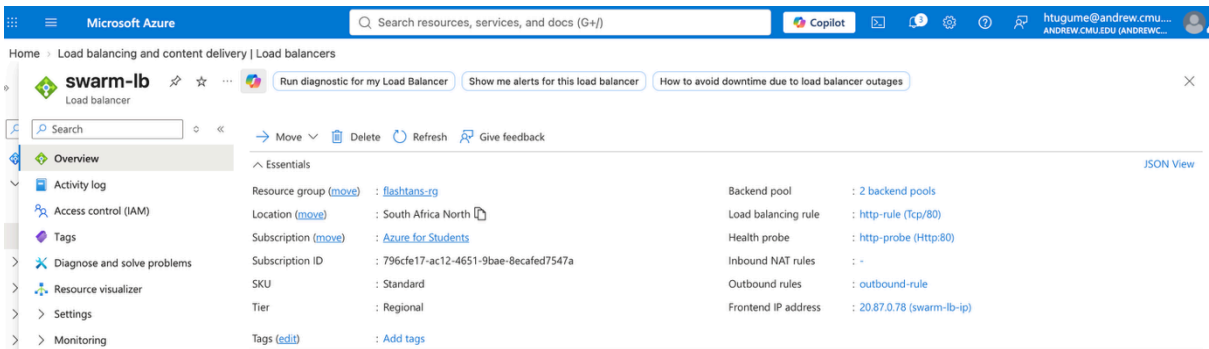


Figure 6: Three-node AKS cluster with two system-pool nodes and one user-pool node, all in Ready state

3.4 Load Balancer and monitoring

A Standard public Azure Load Balancer distributes port 80 traffic to the Swarm nodes, with an HTTP health probe. An outbound rule provides outbound-only internet access to swarm-db. Azure Monitor collects CPU and memory metrics, with alerts configured for high CPU and low available memory.



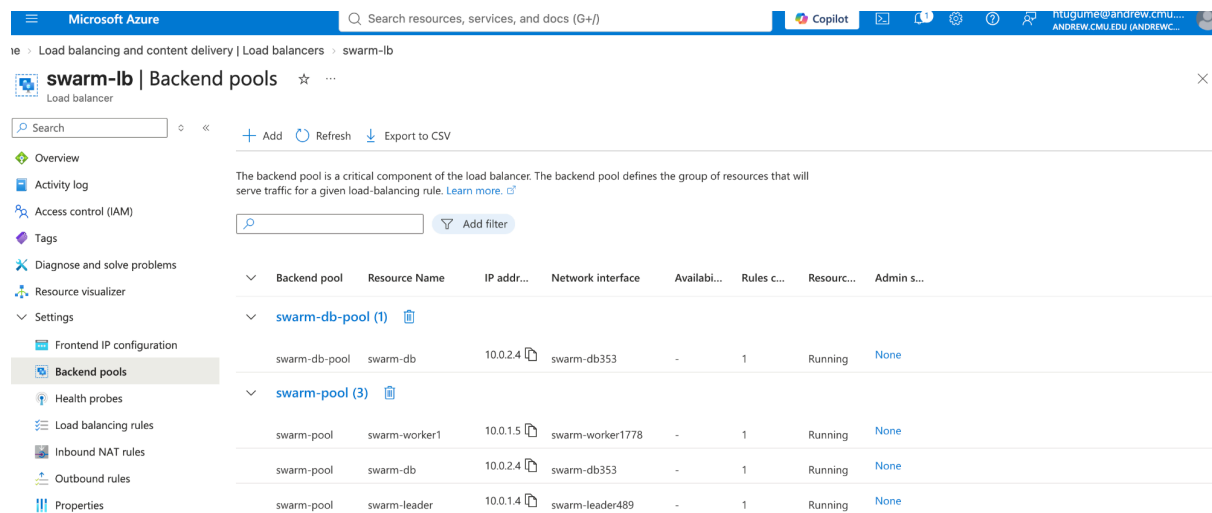


Figure 7: swarm-lb overview and backend pools

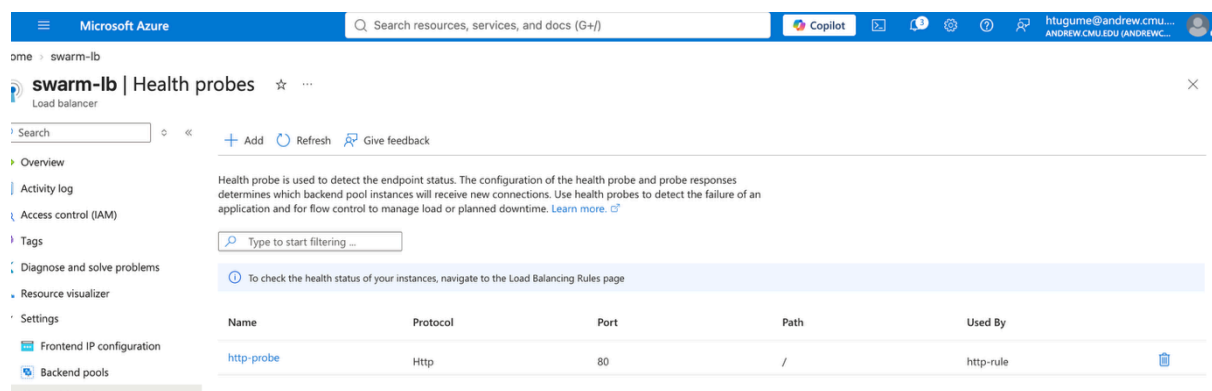


Figure 8: swarm-lb HTTP health probe on port 80

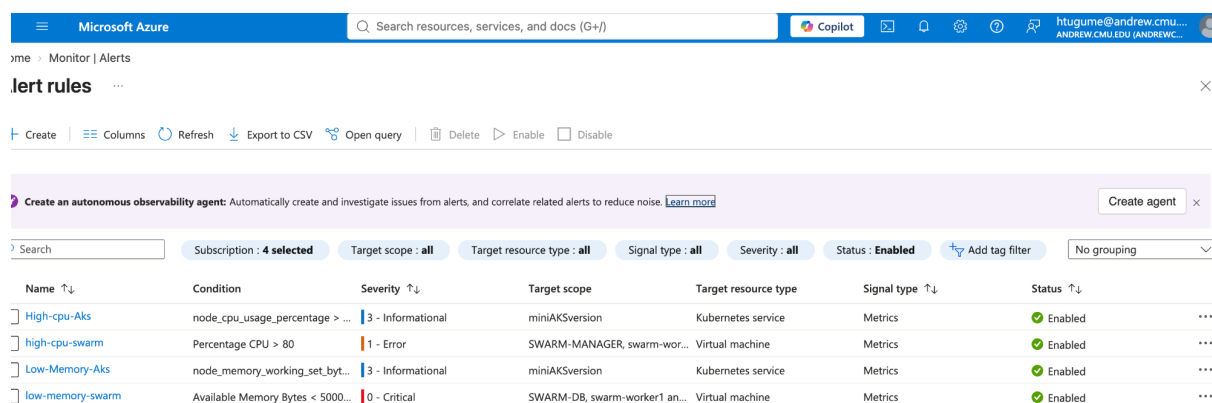


Figure 9: Azure Monitor alert rules

3.5 VNet peering

Because the AKS cluster is in a separate subscription and VNet, the two VNets were peered so that resources communicate over private IPs. Connectivity was verified by pinging an AKS node from swarm-manager.

```
your Cloud Shell session will be ephemeral, so no files or system changes will persist beyond your current session.
davy [ ~ ]$ az account set --subscription 796cfe17-ac12-4651-9bae-8ecafed7547a

az network vnet peering list \
  -g flashtans-rg \
  --vnet-name flashtans-vnet \
  --query "[].(Name:name,State:peeringState,Remote:remoteVirtualNetwork.id)" \
  -o table
Name                               State  Remote
-----
flashtans                          Connected /subscriptions/1d016456-ea73-4f2e-b83f-51c504acac26/resourceGroups/htugume/providers/Microsoft.Network/virtualNetworks/vnet-southafricanorth
flashtans-to-aks                    Connected /subscriptions/1d016456-ea73-4f2e-b83f-51c504acac26/resourceGroups/davymbun/providers/Microsoft.Network/virtualNetworks/aks-vnet
flashtans-to-miniaks                Connected /subscriptions/21bcc171-0f2a-4b21-8c58-646143030955/resourceGroups/MC_Flashtans_miniAKSVersion_southafricanorth/providers/Microsoft.Network/virtualNetworks/aks-vnet-32274588
davy [ ~ ]$ az account set --subscription 21bcc171-0f2a-4b21-8c58-646143030955

az network vnet peering list \
  -g MC_Flashtans_miniAKSVersion_southafricanorth \
  --vnet-name aks-vnet-32274588 \
  --query "[].(Name:name,State:peeringState,Remote:remoteVirtualNetwork.id)" \
  -o table
Name                               State  Remote
-----
miniaks-to-flashtans                Connected /subscriptions/796cfe17-ac12-4651-9bae-8ecafed7547a/resourceGroups/flashtans-rg/providers/Microsoft.Network/virtualNetworks/Flashtans-vnet
davy [ ~ ]$
```

Figure 10: VNet peering between flashtans-vnet and the miniAKSVersion AKS VNet in Connected state

```
Last login: Wed Oct  7 13:07:51 2026 from 41.216.98.178
flashtans@swarm-leader:~$ ping -c 4 10.224.0.5
PING 10.224.0.5 (10.224.0.5) 56(84) bytes of data.
64 bytes from 10.224.0.5: icmp_seq=1 ttl=64 time=4.74 ms
64 bytes from 10.224.0.5: icmp_seq=2 ttl=64 time=1.52 ms
64 bytes from 10.224.0.5: icmp_seq=3 ttl=64 time=0.809 ms
64 bytes from 10.224.0.5: icmp_seq=4 ttl=64 time=0.902 ms

--- 10.224.0.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3047ms
rtt min/avg/max/mdev = 0.809/1.991/4.736/1.607 ms
flashtans@swarm-leader:~$
```

Figure 11: Private connectivity between the Swarm and AKS networks

4. Part 2: Optimizing the Application Container

4.1 Multi-stage Dockerfile

The provided template was improved as follows:

- Upgraded the base image from `node:18-alpine` to `node:22-alpine` to reduce known vulnerabilities.
- Installed `curl` in the production stage, which the template's HEALTHCHECK requires but Alpine does not include.
- Added a `/health` endpoint to the application for the health check.
- Runs as the non-root `node` user and uses a `.dockerignore` to keep the image small.

4.2 Build and push

```
docker build -t sifen23/kane-flashtans-app:latest --push .
```

```
PS C:\Users\hp\flashtans-project\flashtans-app> docker build -t sifen23/kane-flashtans-app:latest --push .
[+] Building 20.3s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 990B
=> [internal] load metadata for docker.io/library/node:22-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 107B
=> [builder 1/5] FROM docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2bc2e402
=> => resolve docker.io/library/node:22-alpine@sha256:0a7108bf6c7bf5de370ffb1a3ed6be93d405b43ff159f681a8d18c0e2bc2e402
=> => extracting sha256:e2de96513ba9eb53b431787ec8a65cddc380ac4772a3e4c4b714dcfde2a102b5
=> => extracting sha256:f7f2d304681aaa935c9cfd180850cf616ae843efce7682873d6521ded7268937
=> => extracting sha256:e554276b05e6306c5ad33cd85bfa7f0693083e21f6114db5abf60a20fb413039
=> => extracting sha256:d39db1cf9caa4f49c5a2e67111f6e0d5eea8ede550cb82f3dc2b79712688d6
=> [internal] load build context
=> => transferring context: 77.13kB
=> [builder 2/5] WORKDIR /app
=> [builder 3/5] COPY package.json yarn.lock ./
=> [builder 4/5] RUN yarn install --production --frozen-lockfile
=> [builder 5/5] COPY . .
=> [stage-1 3/3] COPY --from=builder /app /app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:4405c52861090ea6add31405ea01a02f87d979b71a96c640f3743b983036dedb
=> => exporting config sha256:098247702eb7dc56bf56777223f94fb4d10698a364572b4b379af524025266e7
=> => exporting attestation manifest sha256:6f79c69a0e5a34b21dc36b75494ceed891107a29ff89a853c4a6432cf8fd0bc5
=> => exporting manifest list sha256:db38dd1132a78c2cd6d398d53bf6c41d2a430ddf1560fcee1addf5b4a957be31
=> => naming to docker.io/sifen23/kane-flashtans-app:latest
=> => pushing layers
=> => pushing manifest for docker.io/sifen23/kane-flashtans-app:latest@sha256:db38dd1132a78c2cd6d398d53bf6c41d2a430ddf1560fcee1addf5b4a957be31
=> [auth] sifen23/kane-flashtans-app:pull,push token for registry-1.docker.io
PS C:\Users\hp\flashtans-project\flashtans-app>
```

Figure 12: Image built successfully

Repositories / kane-flashtans-app / General

sifen23/kane-flashtans-app

Last pushed 5 minutes ago · Repository size: 60.8 MB · ☆0 · ↓50

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Tags

DOCKER SCOUT INACTIVE

Activate

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	5 minutes

Figure 13: Image pushed to Docker Hub

5. Part 3: Simulating the Stack with Docker Compose

5.1 docker-compose.yml

The Compose file defines the application service and a single MySQL 8.0 instance with a named volume. The app waits for MySQL's health check to pass before starting (depends_on with condition: service_healthy).



```
PS C:\Users\hp\flashtans-project\flashtans-app> docker compose up -d
[+] up 3/3
✓ Network flashtans-app_default Created 0.0s
✓ Container flashtans-app-mysql-1 Healthy 5.9s
✓ Container flashtans-app-app-1 Started 6.2s
PS C:\Users\hp\flashtans-project\flashtans-app> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
0b3abd787b52   flashtans-app:local  "docker-entrypoint.s..." 43 seconds ago Up 37 seconds (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
48ae6452dbab   mysql:8.0      "docker-entrypoint.s..." 43 seconds ago Up 42 seconds (healthy) 3306/tcp, 33060/tcp
```

Figure 14: Application and MySQL running with Docker Compose

5.2 Launch and persistence test

Data was added through the browser; the stack was stopped with docker compose down and restarted with docker compose up. The data persisted because it is stored in a named volume.

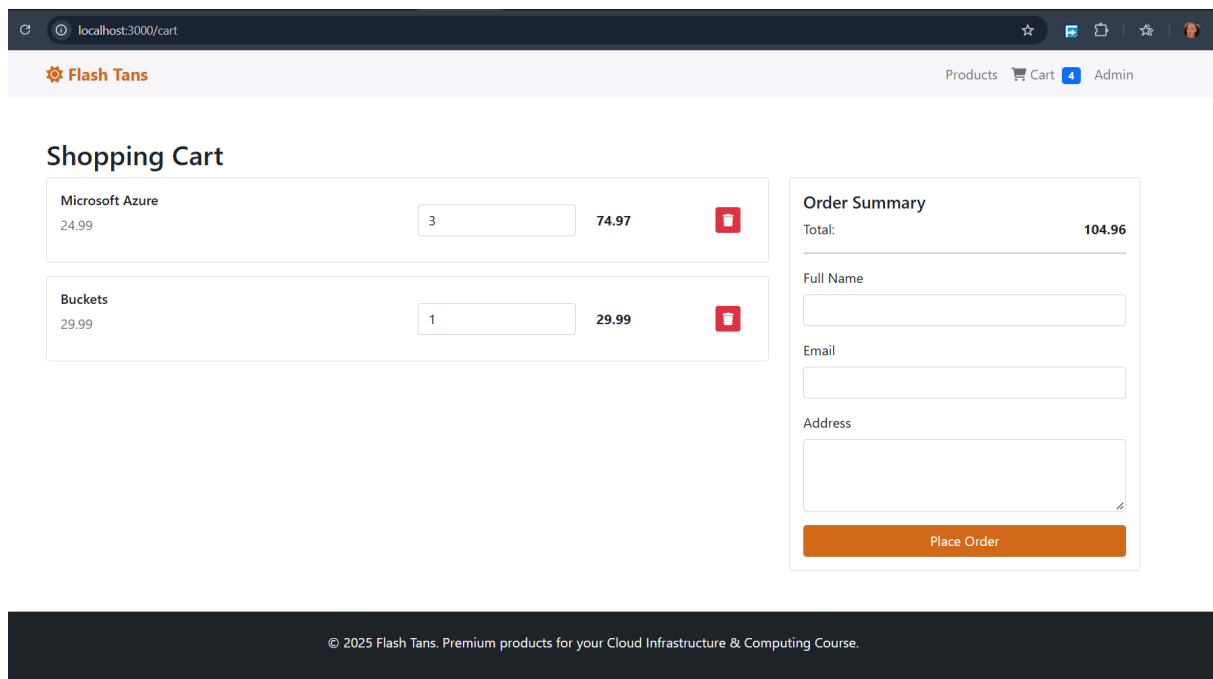


Figure 15: Data added before restarting the stack

```

PS C:\Users\hp\flashtans-project\flashtans-app> docker compose up -d
[+] up 3/3
✓ Network flashtans-app_default Created 0.0s
✓ Container flashtans-app-mysql-1 Healthy 5.9s
✓ Container flashtans-app-app-1 Started 6.2s
PS C:\Users\hp\flashtans-project\flashtans-app> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
0b3abd787b52   flashtans-app:local                 "docker-entrypoint.s..." 43 seconds ago Up 37 seconds (healthy) 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
flashtans-app-app-1
48ae6452dbab   mysql:8.0                           "docker-entrypoint.s..." 43 seconds ago Up 42 seconds (healthy) 3306/tcp, 33060/tcp
flashtans-app-mysql-1
PS C:\Users\hp\flashtans-project\flashtans-app> docker compose down
[+] down 3/3
✓ Container flashtans-app-app-1 Removed 3.4s
✓ Container flashtans-app-mysql-1 Removed 1.7s
✓ Network flashtans-app_default Removed 0.3s
PS C:\Users\hp\flashtans-project\flashtans-app> docker compose up -d
[+] up 3/3
✓ Network flashtans-app_default Created 0.1s
✓ Container flashtans-app-mysql-1 Healthy 6.0s
✓ Container flashtans-app-app-1 Started 6.1s
PS C:\Users\hp\flashtans-project\flashtans-app> docker volume ls
DRIVER   VOLUME NAME
local    0b6bbf34f49e9c891aff5ed46be88c72736890e21d26af79339afc6e78c8d73
local    2b28d084fec9226cc537bab698d5b18ff26109434e143a9c8796e0d2ec5087ec
local    3829014e1f4203a5c24abd6e11e7eed3666388ba99054650d1bfd3bf18bc83b
local    b8b431f25d3dc35313e83905e1a48bbdedaa4b926c39862ccdc94d047859349
local    flashtans-app_mysql_data
local    flashtans-project_mysql_data
local    my-todo-list-lab_pg-data
PS C:\Users\hp\flashtans-project\flashtans-app> $

```

Figure 16: Stack restarted, with the named volume preserved

The screenshot shows a web browser at localhost:3000/cart. The page has a header with the 'Flash Tans' logo and navigation links for 'Products', 'Cart' (with a 4-item icon), and 'Admin'. The main content area is titled 'Shopping Cart' and contains two items:

Item	Price	Quantity	Total	Action
Microsoft Azure	24.99	3	74.97	
Buckets	29.99	1	29.99	

To the right of the cart is an 'Order Summary' section:

- Total:** 104.96
- Full Name:**
- Email:**
- Address:**

Figure 17: Data persisted after restart

6. Part 4: Deploying with Docker Swarm on Azure

6.1 Initializing the swarm

swarm-manager (hostname swarm-leader) was initialized as the manager using its private IP, and swarm-worker1 and swarm-db joined as workers.

`docker swarm init`

```
flashtans@swarm-leader:~$ docker swarm init
Swarm initialized: current node (jrembl1jp0db1av837ddctuqiv) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-006zadxikxgajkojw8hekagrwa0afa6effdeel4qrarrlg2twb-dq5lbfcdpw04c7t8sfhygfcms 10.0.1.4:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

```
flashtans@swarm-leader: ~  Command Prompt
flashtans@swarm-leader:~$ docker node ls
ID                                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
jz6hvbil9sssmc6ekuouf65vl        swarm-db        Ready     Active
jrembl1jp0db1av837ddctuqiv *     swarm-leader    Ready     Active           Leader             29.8.2
807ws2v5cgi3afpgjvcmkj8j0        swarm-worker1    Ready     Active           29.8.1
```

Figure 18: Swarm initialization and three-node docker swarm cluster

6.2 Adapting the Compose file for Swarm

The Compose file was extended with a deploy key (2 app replicas, rolling update configuration and restart policy), an overlay network (flashtans-network) spanning all nodes, a placement constraint pinning MySQL to the database node, and two external Docker secrets (db_root_password and db_password) created on the manager with `docker secret create`. MySQL reads them through `MYSQL_ROOT_PASSWORD_FILE` and `MYSQL_PASSWORD_FILE`, so no password appears in the Compose file. The Swarm version is the `docker-compose.yml` at the project root; docker stack deploy ignores the build key, which explains the “Ignoring unsupported options: build” message in the deployment output.

```
docker-compose.yml
2  services:
35  mysql:
40  volumes:
48  environment:
49  MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
50  MYSQL_DATABASE: ${DB_NAME:-flashtans_db}
51  MYSQL_USER: ${DB_USER:-flashtans}
52  MYSQL_PASSWORD_FILE: /run/secrets/db_password
53  secrets:
54  - db_root_password
55  - db_password
56  healthcheck:
57  test:
58  - CMD-SHELL
59  - MYSQL_PWD="$(cat /run/secrets/db_password)" mysql --protocol=TCP -h 127.0.0.1 -u "$$MYSQL_USER" -D "$$MYSQL_DATABASE" -e 'SELECT 1'
60  interval: 10s
61  timeout: 5s
62  retries: 10
63  start_period: 30s
64
65  volumes:
66  mysql_data:
67
68  networks:
69  flashtans-network:
70  driver: overlay
71
72  secrets:
73  db_root_password:
74  external: true
75  db_password:
76  external: true
```

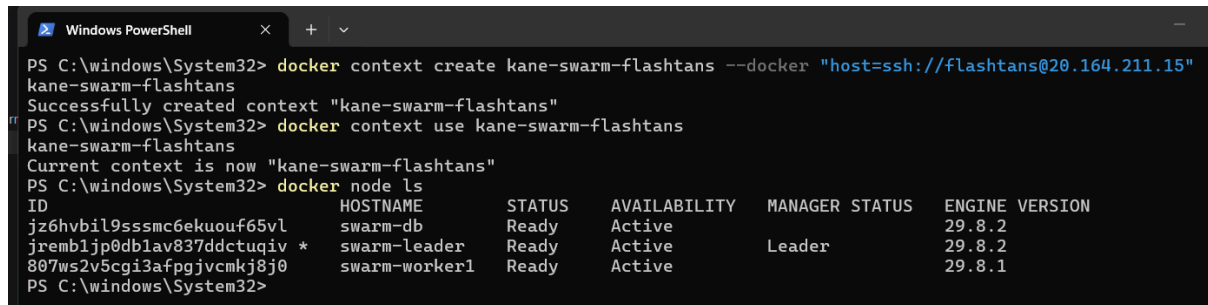
Figure 19: Swarm Compose file with Docker secrets, overlay network and MySQL healthcheck

6.3 Deploying from the local Docker client

docker context create kane-swarm-flashtans --docker "host=ssh://flashtans@20.164.211.15"

docker context use kane-swarm-flashtans

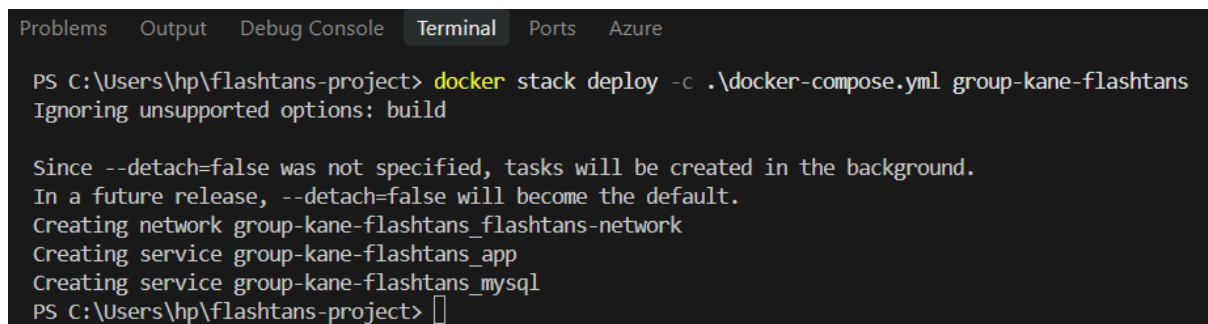
docker stack deploy -c docker-compose.yml group-kane-flashtans



```
PS C:\windows\System32> docker context create kane-swarm-flashtans --docker "host=ssh://flashtans@20.164.211.15"
kane-swarm-flashtans
Successfully created context "kane-swarm-flashtans"
PS C:\windows\System32> docker context use kane-swarm-flashtans
kane-swarm-flashtans
Current context is now "kane-swarm-flashtans"
PS C:\windows\System32> docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
jz6hvbil9ssmc6ekuouf65vl	swarm-db	Ready	Active		29.8.2
jrembljp0db1av837ddctuqiv *	swarm-leader	Ready	Active	Leader	29.8.2
807ws2v5cgi3afpgjvcmkj8j0	swarm-worker1	Ready	Active		29.8.1

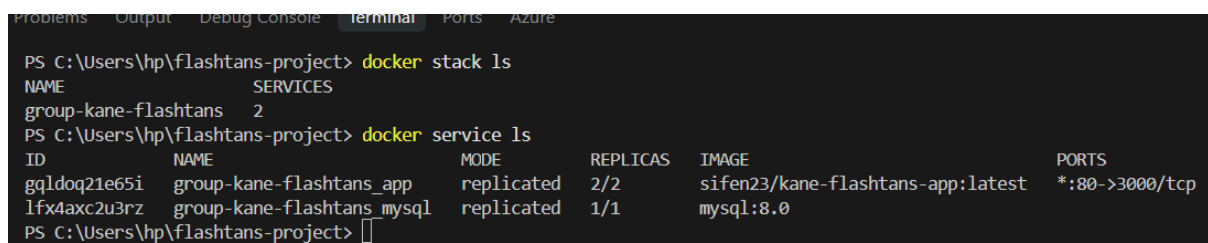
```
PS C:\windows\System32>
```



```
PS C:\Users\hp\flashtans-project> docker stack deploy -c .\docker-compose.yml group-kane-flashtans
Ignoring unsupported options: build

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network group-kane-flashtans_flashtans-network
Creating service group-kane-flashtans_app
Creating service group-kane-flashtans_mysql
PS C:\Users\hp\flashtans-project>
```

Figure 20: Switch docker context and stack deployed from the local Docker client



```
PS C:\Users\hp\flashtans-project> docker stack ls
```

NAME	SERVICES
group-kane-flashtans	2

```
PS C:\Users\hp\flashtans-project> docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
gqldoq21e65i	group-kane-flashtans_app	replicated	2/2	sifen23/kane-flashtans-app:latest	*:80->3000/tcp
lfx4axc2u3rz	group-kane-flashtans_mysql	replicated	1/1	mysql:8.0	

```
PS C:\Users\hp\flashtans-project>
```

Figure 21: Swarm services running

6.4 Load balancer access

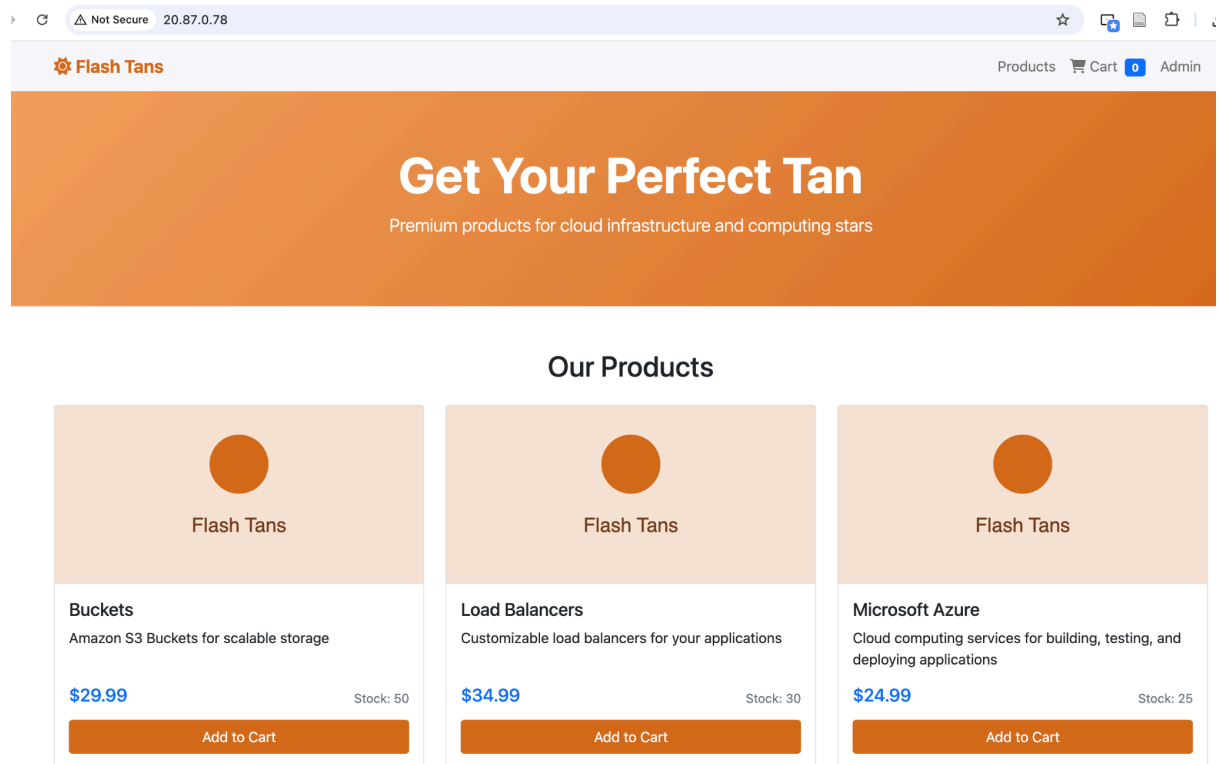


Figure 22: Application accessed through the Azure Load Balancer

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

sgelbrenu@andrew.cmu.edu... ANDREW.CMU.EDU (ANDREW.CMU.EDU)

Home > Load balancing and content delivery > Load balancers

Load balancing and content delivery

andrew.cmu.edu | andrew.cmu.edu (andrew.cmu.onmicrosoft.com)

Search

+ Create

Overview

Load balancing

Application gateways

Load balancers

Content delivery

DNS load balancing

Related services

Help

Name

swarm-lb

Load balancing rule health status

http-rule

The table below provides an overview of the current status of this load balancing rule. Each backend instance's health is indicated by its State and Reason. [Learn more](#)

Properties

Protocol/PortTCP/80

IP address count3

Health probehttp-probe

Overall state100% of instances are healthy

Refresh

If load balancing rule lacks associated health probes or if the virtual machine is not operational, no information will be displayed regarding the load balancing rule state.

Search

State : all

Resource Name	IP Address	State	Reason
swarm-worker1	10.0.1.5	Up	The backend instance is responding to health probe successfully.
swarm-db	10.0.2.4	Up	The backend instance is responding to health probe successfully.
swarm-leader	10.0.1.4	Up	The backend instance is responding to health probe successfully.

Showing 1 of 1

Close

Figure 23: Healthy Swarm backends

7. Part 5: High Availability with Kubernetes and MySQL Replication

7.1 MySQL Group Replication

A three-node, self-healing MySQL cluster was built with a StatefulSet, which gives each replica a stable network name (mysql-0, mysql-1, mysql-2) and its own persistent volume. Group Replication is enabled through a my.cnf stored in a ConfigMap; an init container sets a unique server_id per pod. The application connects through the mysql Service, which selects the primary pod (mysql-0). Members rejoin the group automatically after a restart because group_replication_start_on_boot is ON.

```
navy_mbuto@Mbuto-PC: ~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ cat k8s/mysql-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: mysql-config
  namespace: flashtans
data:
  common.cnf: |
    [mysqld]
    bind-address=0.0.0.0

    gtid_mode=ON
    enforce_gtid_consistency=ON
    log_bin=mysql-bin
    binlog_format=ROW
    transaction_write_set_extraction=XXHASH64

    plugin_load_add=group_replication.so

    loose-group_replication_group_name="aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee"
    loose-group_replication_start_on_boot=ON
    loose-group_replication_recovery_get_public_key=ON
    loose-group_replication_bootstrap_group=OFF
    loose-group_replication_group_seeds="mysql-0.mysql-headless.flashtans.svc.cluster.local:33061,mysql-1.mysql-headless.flashtans.svc.cluster.local:33061,mysql-2.mysql-headless.flashtans.svc.cluster.local:33061"
    loose-group_replication_ip_allowlist="AUTOMATIC"
```

Figure 24: MySQL Group Replication configuration stored in the Kubernetes ConfigMap

```
navy_mbuto@Mbuto-PC: ~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl exec -n flashtans mysql-0 -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SET GLOBAL group_replication_bootstrap_group=ON;
START GROUP_REPLICATION;
SET GLOBAL group_replication_bootstrap_group=OFF;

SELECT
  MEMBER_HOST,
  MEMBER_STATE,
  MEMBER_ROLE
FROM performance_schema.replication_group_members;
"
'
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-0.mysql-headless.flashtans.svc.cluster.local      ONLINE      PRIMARY
```

Figure 25: Bootstrapping the replication group on mysql-0

```
navy_mbuto@Mbuto-PC: ~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ for p in mysql-1 mysql-2; do
echo "===== $p secondary status ====="
kubectl exec -n flashtans "$p" -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SELECT
  MEMBER_HOST,
  MEMBER_STATE,
  MEMBER_ROLE
FROM performance_schema.replication_group_members
WHERE MEMBER_HOST = @@report_host;
"
'
done
===== mysql-1 secondary status =====
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-1.mysql-headless.flashtans.svc.cluster.local      ONLINE      SECONDARY
===== mysql-2 secondary status =====
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-2.mysql-headless.flashtans.svc.cluster.local      ONLINE      SECONDARY
navy_mbuto@Mbuto-PC: ~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$
```

Figure 26: mysql-1 and mysql-2 successfully joined the MySQL Group Replication cluster as ONLINE secondary members

```

davy_mbuto@Mbuto-PC:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl exec -n flashtans mysql-0 -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SELECT
  MEMBER_HOST,
  MEMBER_STATE,
  MEMBER_ROLE
FROM performance_schema.replication_group_members;
"
'
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-0.mysql-headless.flashtans.svc.cluster.local      ONLINE PRIMARY
mysql-2.mysql-headless.flashtans.svc.cluster.local      ONLINE SECONDARY
mysql-1.mysql-headless.flashtans.svc.cluster.local      ONLINE SECONDARY

```

Figure 27: Three-member MySQL Group Replication cluster with one PRIMARY and two ONLINE SECONDARY members

```

davy_mbuto@Mbuto-PC:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ echo "==== MYSQL SERVICE ====="
kubectl get svc mysql -n flashtans -o wide

echo
echo "==== SERVICE SELECTOR ====="
kubectl get svc mysql -n flashtans \
  -o jsonpath='{.spec.selector}{\n\''}'

echo
echo "==== MYSQL SERVICE ENDPOINT ====="
kubectl get endpointslice -n flashtans \
  -l kubernetes.io/service-name=mysql

echo
echo "==== CURRENT MYSQL PRIMARY ====="
kubectl exec -n flashtans mysql-0 -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SELECT MEMBER_HOST, MEMBER_STATE, MEMBER_ROLE
FROM performance_schema.replication_group_members
WHERE MEMBER_ROLE='\''PRIMARY'\'';
"
'

==== MYSQL SERVICE =====
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE      SELECTOR
mysql     ClusterIP  10.98.68.80      <none>           3306/TCP      32h      statefulset.kubernetes.io/pod-name=mysql-0

==== SERVICE SELECTOR =====
{"statefulset.kubernetes.io/pod-name":"mysql-0"}

==== MYSQL SERVICE ENDPOINT =====
NAME      ADDRESSSTYPE      PORTS      ENDPOINTS      AGE
mysql-1x45c  IPv4              3306       10.244.0.6      32h

==== CURRENT MYSQL PRIMARY =====
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-0.mysql-headless.flashtans.svc.cluster.local      ONLINE PRIMARY

```

Figure 28: The mysql Service routing application traffic to the primary (mysql-0)

7.2 Kubernetes manifests

File	Object	Purpose
secret.yaml	Secret	MySQL root, replication and application passwords
mysql-configmap.yaml	ConfigMap	my.cnf enabling Group Replication
mysql-headless-svc.yaml	Headless Service	DNS discovery between MySQL pods
mysql-statefulset.yaml	StatefulSet	Three MySQL replicas with persistent storage
mysql-router.yaml	Service	ClusterIP Service mysql selecting the primary pod (mysql-0)
app-deployment.yaml	Deployment	Stateless Node.js application pods
app-service.yaml	Service (LoadBalancer)	Exposes the application to the internet
app-hpa.yaml	HorizontalPodAutoscaler	Scales the app between 3 and 10 pods at 70% CPU

7.3 Local deployment on Minikube

kubectl apply -f .

```

davy_mbutu@Mbutu-PC:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl get pods,svc,hpa -n flashtans

```

NAME	READY	STATUS	RESTARTS	AGE
pod/flashtans-app-5b5f5f85b9-czrwj	1/1	Running	1 (32m ago)	28h
pod/flashtans-app-5b5f5f85b9-jwd7x	1/1	Running	1 (32m ago)	29h
pod/flashtans-app-5b5f5f85b9-mj795	1/1	Running	1 (32m ago)	29h
pod/mysql-0	1/1	Running	1 (32m ago)	31h
pod/mysql-1	1/1	Running	1 (32m ago)	28h
pod/mysql-2	1/1	Running	1 (32m ago)	29h

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/flashtans-app	NodePort	10.99.97.74	<none>	80:31418/TCP	29h
service/mysql	ClusterIP	10.98.68.80	<none>	3306/TCP	31h
service/mysql-headless	ClusterIP	None	<none>	3306/TCP,33061/TCP	31h

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/flashtans-app	Deployment/flashtans-app	cpu: 1%/70%	3	10	3	28h

Figure 29: Full Flash Tans stack running successfully on local Minikube Kubernetes

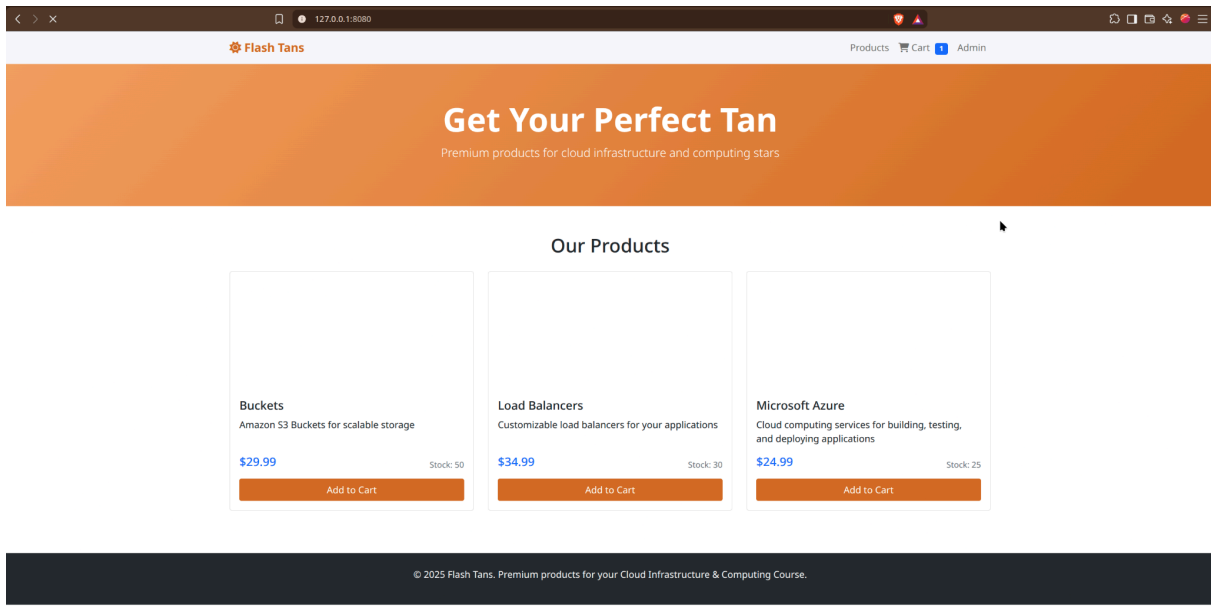


Figure 30: Flash Tans application running successfully on local Kubernetes

7.4 Migration to AKS

kubectl was connected to AKS and the same manifests were applied, demonstrating Kubernetes' portability. The only difference is the application Service type: NodePort on Minikube, which has no cloud load balancer, and LoadBalancer on AKS. Replication was bootstrapped again on the new cluster.

```
az aks get-credentials -g Flashtans -n miniAKSversion
```

```
kubectl apply -f .
```

```
^Cdav [ ~/flashtans-project ] kubectl get pods,svc,hpa -n flashtans
NAME                                READY   STATUS    RESTARTS   AGE
pod/flashtans-app-65994d74d5-6pnb7  1/1     Running   0           114s
pod/flashtans-app-65994d74d5-qnzwf  1/1     Running   0           114s
pod/flashtans-app-65994d74d5-tnvpc  1/1     Running   0           114s
pod/mysql-0                          1/1     Running   0           29m
pod/mysql-1                          1/1     Running   0           20m
pod/mysql-2                          1/1     Running   0           15m

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/flashtans-app               LoadBalancer  10.0.97.10    20.87.193.174 80:32043/TCP     111s
service/mysql                       ClusterIP      10.0.41.144   <none>        3306/TCP         52m
service/mysql-headless              ClusterIP      None          <none>        3306/TCP,33061/TCP 52m

NAME                                REFERENCE             TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/flashtans-app  Deployment/flashtans-app  cpu: 2%/70%     3         10        3           113s
dav [ ~/flashtans-project ] $
```

```

davy [ ~ ]$ echo "=== CURRENT CONTEXT ==="
kubectrl config current-context
echo
kubectrl get pods,svc,hpa -n flashtans
=== CURRENT CONTEXT ===
miniAKSversion

NAME                                READY   STATUS    RESTARTS   AGE
pod/ab-video                         1/1     Running   0           3h49m
pod/flashtans-app-65994d74d5-69j79  1/1     Running   0           3h48m
pod/flashtans-app-65994d74d5-gbgzc  1/1     Running   0           3h46m
pod/flashtans-app-65994d74d5-glzzc  1/1     Running   0           24h
pod/flashtans-app-65994d74d5-j2t4m  1/1     Running   0           3h48m
pod/flashtans-app-65994d74d5-lgzvs  1/1     Running   0           24h
pod/flashtans-app-65994d74d5-mgwg7  1/1     Running   0           3h49m
pod/flashtans-app-65994d74d5-qf7ws  1/1     Running   0           3h48m
pod/flashtans-app-65994d74d5-rskbn  1/1     Running   0           3h48m
pod/flashtans-app-65994d74d5-spxms  1/1     Running   0           3h48m
pod/flashtans-app-65994d74d5-tnvpc  1/1     Running   0           25h
pod/mysql-0                         1/1     Running   0           25h
pod/mysql-1                         1/1     Running   0           25h
pod/mysql-2                         1/1     Running   0           25h

NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/flashtans-app               LoadBalancer        10.0.97.10      20.87.193.174   80:32043/TCP     25h
service/mysql                       ClusterIP            10.0.41.144     <none>           3306/TCP          25h
service/mysql-headless              ClusterIP            None            <none>           3306/TCP,33061/TCP 25h

NAME                                REFERENCE           TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
horizontalpodautoscaler.autoscaling/flashtans-app  Deployment/flashtans-app  cpu: 273%/70%    3         10        10         25h

```

Figure 31: Full Flash Tans stack running on AKS with the application exposed through a public LoadBalancer

```

davy [ ~/flashtans-project ]$ curl -I http://20.87.193.174/
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 7182
ETag: W/"1c0e-3IHxXqzleLxDP+9L2uKffgZUbik"
Date: Thu, 08 Oct 2026 13:51:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```

Figure 32: AKS public endpoint returning HTTP 200

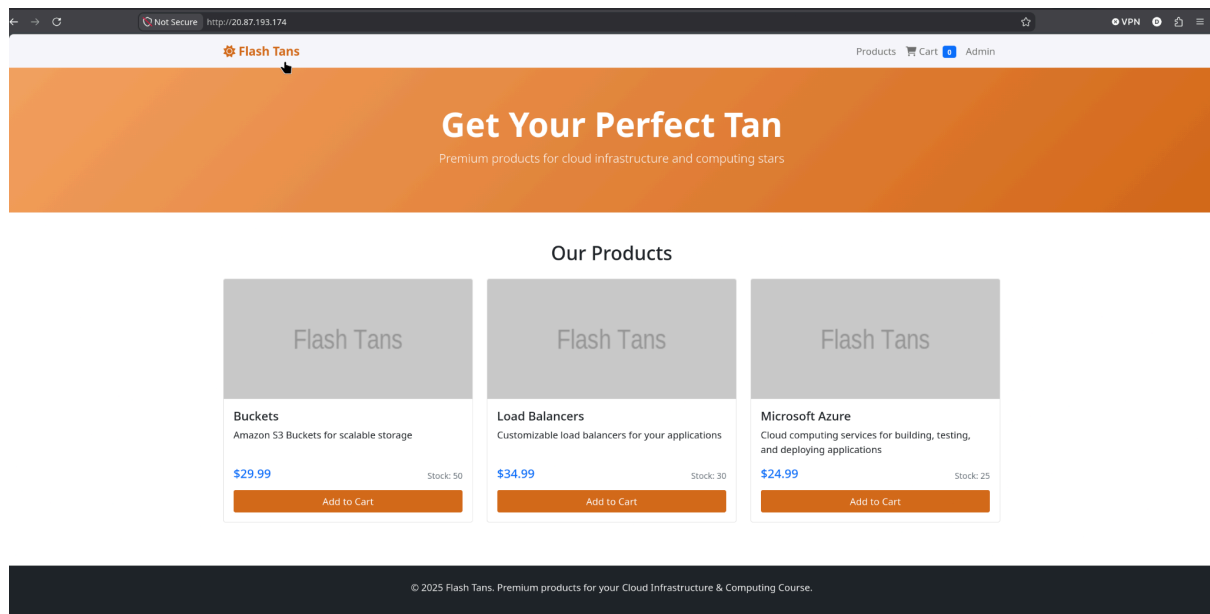


Figure 33: Live application on AKS

7.5 Resilience testing: the flash sale simulation

Load test. Each resilience test was first rehearsed on the local Minikube cluster and then repeated on AKS. Both sets of results are shown below; the AKS screenshots show the miniAKSversion context. Apache Bench was run from a pod inside the AKS cluster against the public IP, and completed all 20,000 requests with 0 failed requests:

```
ab -n 20000 -c 500 http://20.87.193.174/
```

```

davy [ ~/flashtans-project ]$ kubectl run ab-load \
-n flashtans \
--rm -it \
--restart=Never \
--image=httpd:2.4-alpine \
-- /usr/local/apache2/bin/ab \
-n 20000 -c 500 \
http://20.87.193.174/
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 20.87.193.174 (be patient)
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
Completed 2000 requests
Completed 4000 requests
Completed 6000 requests
Completed 8000 requests
Completed 10000 requests
Completed 12000 requests
Completed 14000 requests
Completed 16000 requests
Completed 18000 requests
Completed 20000 requests
Finished 20000 requests

Server Software:
Server Hostname:      20.87.193.174
Server Port:          80

Document Path:        /
Document Length:       7182 bytes

Concurrency Level:     500
Time taken for tests:   29.934 seconds
Complete requests:      20000
Failed requests:         0
Total transferred:      147720000 bytes
HTML transferred:       143640000 bytes
Requests per second:    668.15 [#/sec] (mean)
Time per request:       748.340 [ms] (mean)
Time per request:       1.497 [ms] (mean, across all concurrent requests)
Transfer rate:          4819.26 [Kbytes/sec] received

Connection Times (ms)
              min      mean[+/-sd] median   max
Connect:        0        3   8.1      1    156
Processing:      2       727  731.4    548   2124
Waiting:         0       710  724.2    518   2100
Total:          2       730  732.3    550   2125

Percentage of the requests served within a certain time (ms)
 50%    550
 66%    908
 75%   1427
 80%   1631
 90%   1949
 95%   1991
 98%   2028
 99%   2062
100%   2125 (longest request)
Session ended, resume using 'kubectl attach ab-load -c ab-load -n flashtans -i -t' command
pod "ab-load" deleted from flashtans namespace

```

Figure 34: Apache Bench load test against the AKS public endpoint

Autoscaling. The HPA scaled the application out as the CPU exceeded the 70% target.

```

davy_mbuto@Mbuto-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl get hpa -n flashtans -w
NAME           REFERENCE                TARGETS      MINPODS   MAXPODS   REPLICAS   AGE
flashtans-app  Deployment/flashtans-app  cpu: 1%/70%    3         10         3          29h
flashtans-app  Deployment/flashtans-app  cpu: 235%/70%  3         10         3          29h
flashtans-app  Deployment/flashtans-app  cpu: 235%/70%  3         10         6          29h
flashtans-app  Deployment/flashtans-app  cpu: 235%/70%  3         10         10         29h
flashtans-app  Deployment/flashtans-app  cpu: 199%/70%  3         10         10         29h
flashtans-app  Deployment/flashtans-app  cpu: 304%/70%  3         10         10         29h

```

Figure 35: Local Minikube rehearsal: HPA scaling out under load

```
< > C portal.azure.com/#home
Microsoft Azure
Switch to PowerShell Restart Manage files New session Editor Open in VS Code for the Web Web preview

Every 5.0s:
=== AKS CONTEXT ===
miniAKSVersion

=== HPA ===
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AC
flashtans-app Deployment/flashtans-app cpu: 188%/70% 3 10 10 25

=== APP PODS ===
NAME READY STATUS RESTARTS AGE
flashtans-app-65994d74d5-69j79 1/1 Running 0 4h11m
flashtans-app-65994d74d5-gbgzc 1/1 Running 0 4h9m
flashtans-app-65994d74d5-glzzc 1/1 Running 1 (3m32s ago) 25h
flashtans-app-65994d74d5-j2t4m 1/1 Running 0 4h11m
flashtans-app-65994d74d5-lgzvs 1/1 Running 0 25h
flashtans-app-65994d74d5-mgwq7 1/1 Running 0 4h12m
flashtans-app-65994d74d5-qf7ws 1/1 Running 0 4h11m
flashtans-app-65994d74d5-rskbn 1/1 Running 0 4h11m
flashtans-app-65994d74d5-spxms 1/1 Running 0 4h11m
flashtans-app-65994d74d5-tnvpc 1/1 Running 0 25h
```

Figure 36: AKS Horizontal Pod Autoscaler scaling the Flash Tans application from 3 to 10 replicas as CPU exceeded the 70% target

Application pod failure. An application pod was deleted during the test; the Deployment replaced it immediately with no interruption.

```
davy_mbutu@Mbutu-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ POD=$(kubectl get pods -n flashtans -l app=flashtans-app \
-o jsonpath='{.items[0].metadata.name}')

echo "Deleting $POD"
kubectl delete pod -n flashtans "$POD"

kubectl get pods -n flashtans -l app=flashtans-app -w
Deleting flashtans-app-5b5f5f85b9-czrwj
pod "flashtans-app-5b5f5f85b9-czrwj" deleted from flashtans namespace
NAME READY STATUS RESTARTS AGE
flashtans-app-5b5f5f85b9-jwd7x 1/1 Running 1 (54m ago) 29h
flashtans-app-5b5f5f85b9-mj795 1/1 Running 1 (54m ago) 29h
flashtans-app-5b5f5f85b9-tfwvh 1/1 Running 0 31s
```

Figure 37: Local Minikube rehearsal: Kubernetes automatically replacing a deleted application pod while the Flash Tans service remains available

```
< > ↺ portal.azure.com/#home
Microsoft Azure
Switch to PowerShell Restart Manage files New session Editor Open in VS Code for the Web Web preview
Deleting application pod: flashtans-app-65994d74d5-69j79
pod "flashtans-app-65994d74d5-69j79" deleted from flashtans namespace
davy [ ~ ]$ echo "=== APPLICATION PODS AFTER FAILURE ==="
kubectl get pods \
  -n flashtans \
  -l app=flashtans-app

echo
echo "=== SERVICE AVAILABILITY DURING RECOVERY ==="
kubectl logs availability-check \
  -n flashtans \
  --tail=25
=== APPLICATION PODS AFTER FAILURE ===
NAME                                READY   STATUS    RESTARTS   AGE
flashtans-app-65994d74d5-5trj6      1/1     Running   0           49s
flashtans-app-65994d74d5-gbgzc      1/1     Running   0           4h12m
flashtans-app-65994d74d5-glzzc      1/1     Running   1 (6m29s ago) 25h
flashtans-app-65994d74d5-j2t4m      1/1     Running   0           4h14m
flashtans-app-65994d74d5-lgzvs      1/1     Running   0           25h
flashtans-app-65994d74d5-mgwq7      1/1     Running   0           4h15m
flashtans-app-65994d74d5-qf7ws      1/1     Running   0           4h14m
flashtans-app-65994d74d5-rskbn      1/1     Running   0           4h14m
flashtans-app-65994d74d5-spxms      1/1     Running   0           4h14m
flashtans-app-65994d74d5-tnvpc      1/1     Running   0           25h

=== SERVICE AVAILABILITY DURING RECOVERY ===
15:14:20 HTTP 200
15:14:22 HTTP 200
15:14:26 HTTP 200
15:14:31 HTTP 200
15:14:32 HTTP 200
15:14:37 HTTP 200
15:14:38 HTTP 200
15:14:39 HTTP 200
15:14:40 HTTP 200
15:14:41 HTTP 200
15:14:42 HTTP 200
15:14:43 HTTP 200
15:14:44 HTTP 200
15:14:45 HTTP 200
15:14:46 HTTP 200
15:14:47 HTTP 200
15:14:54 HTTP 200
15:14:55 HTTP 200
15:15:00 HTTP 200
15:15:02 HTTP 200
15:15:03 HTTP 200
15:15:06 HTTP 200
15:15:07 HTTP 200
15:15:10 HTTP 200
15:15:11 HTTP 200
davy [ ~ ]$
```

Figure 38: AKS automatically replacing a deleted application pod while the Flash Tans service remains available

Database pod failure. mysql-1 was deleted; the application remained online, and the StatefulSet recreated the pod, which rejoined the replication group.

```
davy_mbuto@Mbuto-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl delete pod mysql-1 -n flashtans
pod "mysql-1" deleted from flashtans namespace
davy_mbuto@Mbuto-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl get pod mysql-1 -n flashtans -w
NAME      READY   STATUS    RESTARTS   AGE
mysql-1   1/1     Running   0           2m17s
davy_mbuto@Mbuto-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl exec -n flashtans mysql-0 -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SELECT
  MEMBER_HOST,
  MEMBER_STATE,
  MEMBER_ROLE
FROM performance_schema.replication_group_members;
"
'
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-0.mysql-headless.flashtans.svc.cluster.local      ONLINE   PRIMARY
mysql-2.mysql-headless.flashtans.svc.cluster.local      ONLINE   SECONDARY
mysql-1.mysql-headless.flashtans.svc.cluster.local      ONLINE   SECONDARY
davy_mbuto@Mbuto-Pc:~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$
```

Figure 39: Local Minikube rehearsal: mysql-1 recovered after pod deletion and rejoined the MySQL Group Replication cluster as an ONLINE secondary

```
davy [ ~ ]$ kubectl get pod mysql-1 -n flashtans -w
NAME      READY   STATUS    RESTARTS   AGE
mysql-1   0/1     Init:0/1   0           12s
mysql-1   0/1     PodInitializing  0           38s
mysql-1   0/1     Running    0           51s
mysql-1   1/1     Running    0           73s
davy [ ~ ]$ echo "=== AKS CONTEXT ==="
kubectl config current-context
echo
echo "=== RECREATED MYSQL POD ==="
kubectl get pod mysql-1 -n flashtans
echo
echo "=== GROUP REPLICATION AFTER RECOVERY ==="
kubectl exec -n flashtans mysql-0 -- sh -c '
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" -e "
SELECT MEMBER_HOST, MEMBER_STATE, MEMBER_ROLE
FROM performance_schema.replication_group_members;
"
'
=== AKS CONTEXT ===
miniAKSVersion

=== RECREATED MYSQL POD ===
NAME      READY   STATUS    RESTARTS   AGE
mysql-1   1/1     Running   0           2m11s

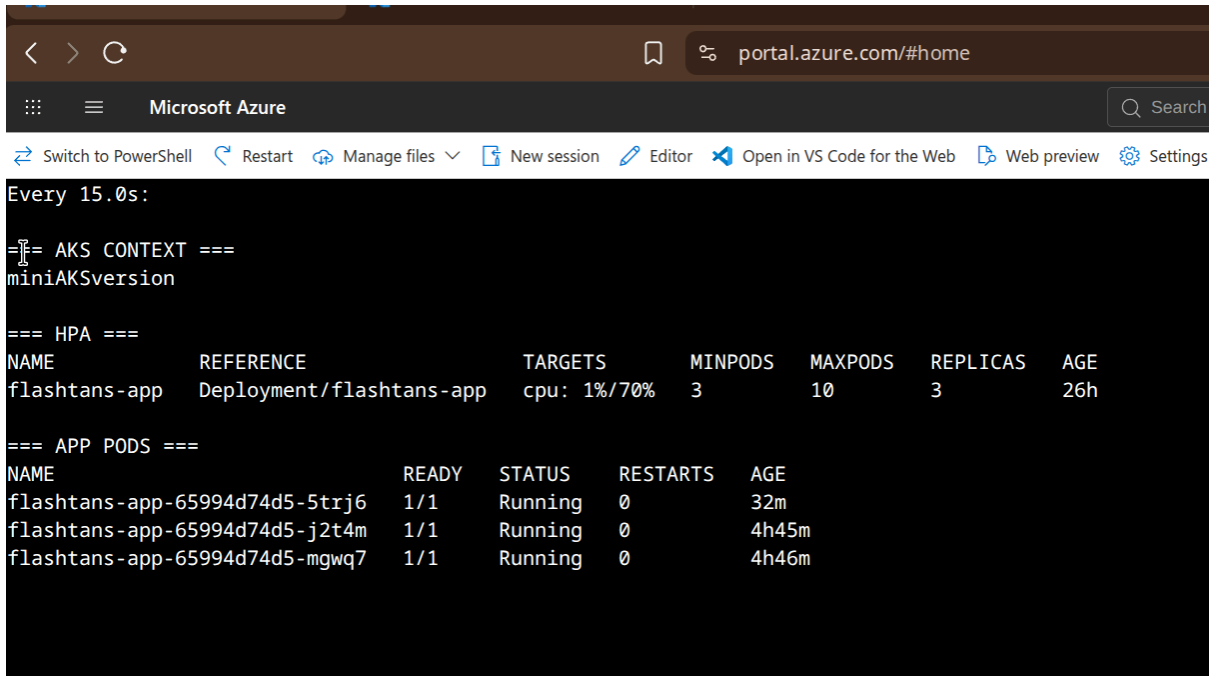
=== GROUP REPLICATION AFTER RECOVERY ===
Defaulted container "mysql" out of: mysql, init-mysql-config (init)
mysql: [Warning] Using a password on the command line interface can be insecure.
MEMBER_HOST      MEMBER_STATE      MEMBER_ROLE
mysql-1.mysql-headless.flashtans.svc.cluster.local      ONLINE   SECONDARY
mysql-0.mysql-headless.flashtans.svc.cluster.local      ONLINE   PRIMARY
mysql-2.mysql-headless.flashtans.svc.cluster.local      ONLINE   SECONDARY
davy [ ~ ]$
```

Figure 40: AKS StatefulSet recreating mysql-1 after pod deletion and automatically restoring it as an ONLINE MySQL Group Replication secondary

Scale-in. After the test, the HPA scaled the application back to the minimum replica count.

```
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 3 29h
davy_mbuto@Mbuto-PC: ~/Desktop/Academia/Cloud Computing/Lab 4/flashtans-project$ kubectl get hpa -n flashtans -w
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 3 29h
flashtans-app Deployment/flashtans-app cpu: 235%/70% 3 10 3 29h
flashtans-app Deployment/flashtans-app cpu: 235%/70% 3 10 6 29h
flashtans-app Deployment/flashtans-app cpu: 235%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 199%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 304%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 368%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 361%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 175%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 3%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 10 29h
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 3 29h
```

Figure 41: Local Minikube rehearsal: HPA scaled back in after the load test



```
Every 15.0s:
== AKS CONTEXT ==
miniAKSVersion

=== HPA ===
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
flashtans-app Deployment/flashtans-app cpu: 1%/70% 3 10 3 26h

=== APP PODS ===
NAME READY STATUS RESTARTS AGE
flashtans-app-65994d74d5-5trj6 1/1 Running 0 32m
flashtans-app-65994d74d5-j2t4m 1/1 Running 0 4h45m
flashtans-app-65994d74d5-mgwq7 1/1 Running 0 4h46m
```

Figure 42: AKS Horizontal Pod Autoscaler scaling the Flash Tans application back to the minimum of three replicas after the load ended

8. Swarm vs. Kubernetes: Deployment Experience

Docker Swarm was quicker to set up because it reused much of the Docker Compose configuration and made it easy to deploy replicated services across the Azure VMs. However, the stateful MySQL service was harder to manage because storage was tied to a specific node and required placement constraints. Kubernetes required more YAML and more initial configuration, but it provided stronger orchestration features such as StatefulSets, persistent volumes, health checks, self-healing, Services, and Horizontal Pod Autoscaling. On AKS, the managed Kubernetes control plane reduced the operational work of maintaining the cluster, while the application could scale automatically during the Apache Bench load test and recover after pod failures. Overall, Swarm was simpler for fast deployment, while Kubernetes and AKS provided better resilience, scaling, and management for a production-style workload.